

Flannel, Canal, Calico & Cilium

Not four equivalent products: understanding what each CNI actually does

Flannel

Network fabric

Canal

Flannel + Calico policy

Calico

Networking + policy

Cilium

eBPF platform

What we will cover

1

The right mental model

Connectivity vs policy vs platform

2

Flannel

Backends and the VXLAN overlay

3

Canal

Flannel networking + Calico policy

4

Calico

BGP, VXLAN, IP-in-IP, eBPF, NetworkPolicy

5

Cilium

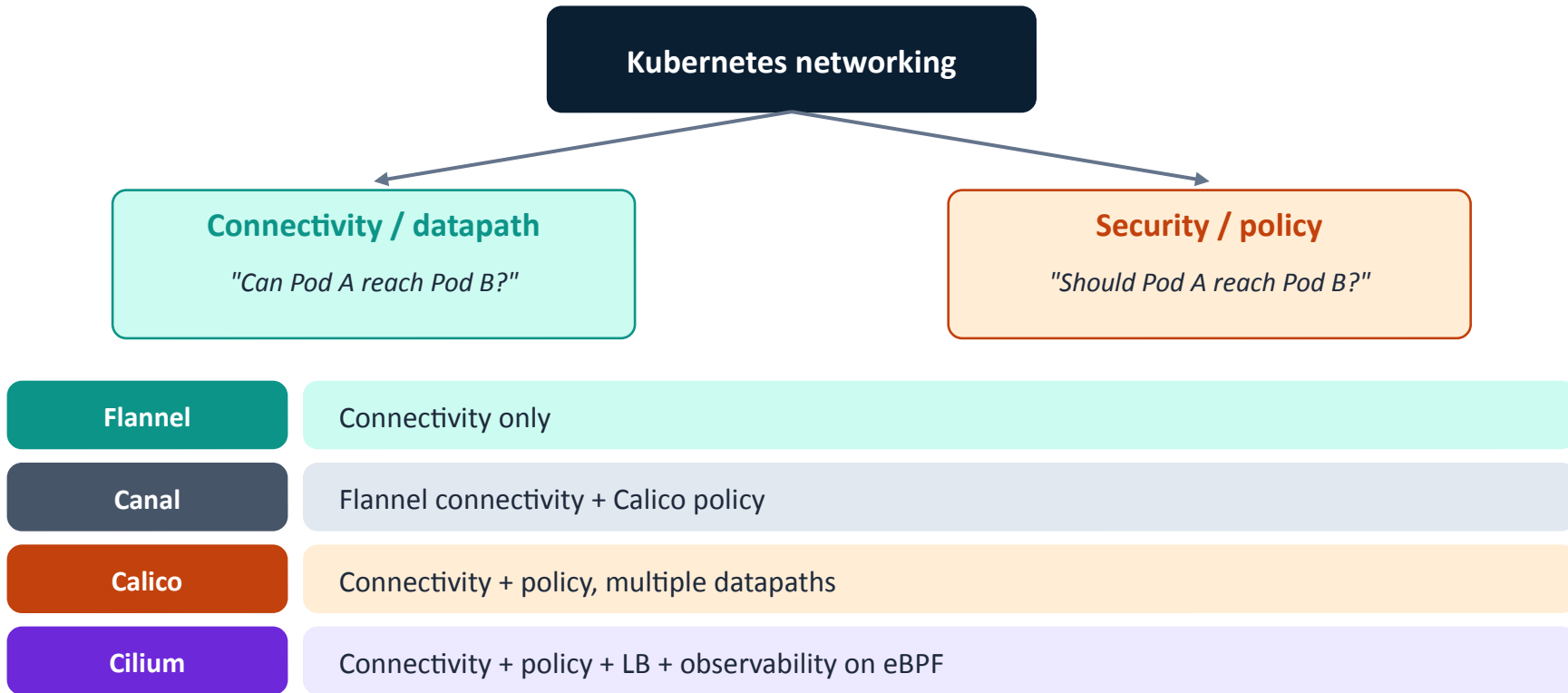
eBPF datapath, kube-proxy replacement, L7, Hubble

6

Side-by-side

Encryption, IPAM, routing, complexity, K3s setup

Different jobs, not different brands



The four CNIs at a glance

Feature	Flannel	Canal	Calico	Cilium
Primary purpose	Pod networking	Networking + policy	Networking + policy	Networking + policy + eBPF
Pod IPAM	host-local	host-local	Calico IPAM	Cilium IPAM
Overlay options	VXLAN	VXLAN (Flannel)	VXLAN, IP-in-IP	VXLAN, Geneve
Native routing / BGP	host-gw (no BGP)	No	Yes, BGP	Native routing; BGP optional
NetworkPolicy	Not by itself	Yes (Calico)	Yes + extended	Yes + extended
L7 policy	No	Limited	Limited	Yes (HTTP, DNS, Kafka)
kube-proxy replacement	No	No	Yes (eBPF mode)	Yes
Flow observability	No	No	Limited (OSS)	Yes (Hubble)
Encryption	WireGuard backend	Via Flannel	WireGuard	WireGuard, IPsec
Complexity	Low	Medium	Medium-High	High

01

Flannel

A simple network fabric, and the K3s default

Pod IPs plus node-to-node connectivity

Backends supported in K3s

vxlan

Default

Wraps Pod traffic in UDP (port 8472). Works on almost any network; the underlay never sees Pod IPs.

host-gw

No encapsulation

Installs plain Linux routes to other nodes. Faster, but nodes must share an L2 segment.

wireguard-native

Encrypted

Tunnels Pod traffic through WireGuard between nodes. Use when node links are untrusted.

Flannel deliberately does NOT try to be:

Firewall

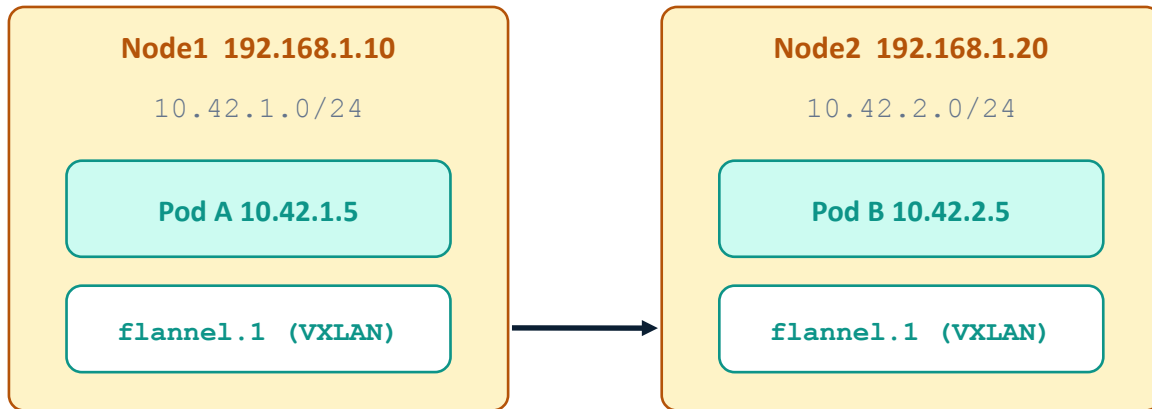
L7 security engine

Service mesh

Observability tool

eBPF platform

The underlay never needs to know Pod IPs



Host network sees

UDP 8472 192.168.1.10 → 192.168.1.20

Inside the VXLAN packet

10.42.1.5 → 10.42.2.5

Why this matters

- No changes to switches or routers
- Works across subnets and most clouds
- Only needs UDP 8472 open between nodes
- Cost: ~50 bytes overhead, lower MTU

02

Canal

Flannel networking + Calico policy

Canal = Flannel + Calico policy



Why it existed

- Flannel was the simplest overlay
- Calico had the best policy engine
- Canal gave you both without running BGP
- Still the default CNI in RKE2

Why it matters less today

- Calico now has native VXLAN
- One component instead of two
- Calico docs recommend Calico VXLAN for new clusters
- Migration path: Canal → Calico

03

Calico

A networking and policy platform with multiple datapaths

One CNI, four ways to move packets

BGP

Native routing

Pod routes distributed via BGP. No encapsulation. Underlay can route Pod IPs.

VXLAN

Overlay

Same overlay behaviour as Flannel VXLAN, plus Calico policy. No BGP needed.

IP-in-IP

Overlay

Lighter overlay (20 B). Needs IP protocol 4 allowed. Often with BGP.

eBPF

Dataplane

Replaces iptables with eBPF programs; can replace kube-proxy.

Also: CrossSubnet mode encapsulates only traffic that crosses a subnet boundary, and routes natively within a subnet: best of both.

Native routing with BGP



```
# Node1 routing table
10.42.2.0/24 via 192.168.1.20 dev eth0 proto bird
```

Packet on the wire: 10.42.1.5 → 10.42.2.5 (no outer header)

Can also peer with top-of-rack routers so the whole datacenter can reach Pod IPs directly.

Trade-offs

Gains

- No encaps overhead, full MTU
- Pod IPs visible to network tools

Costs

- Network must accept Pod routes
- Needs BGP knowledge

NetworkPolicy: who may talk to whom



Selecting a Pod makes it default-deny for that direction; only listed peers get in.

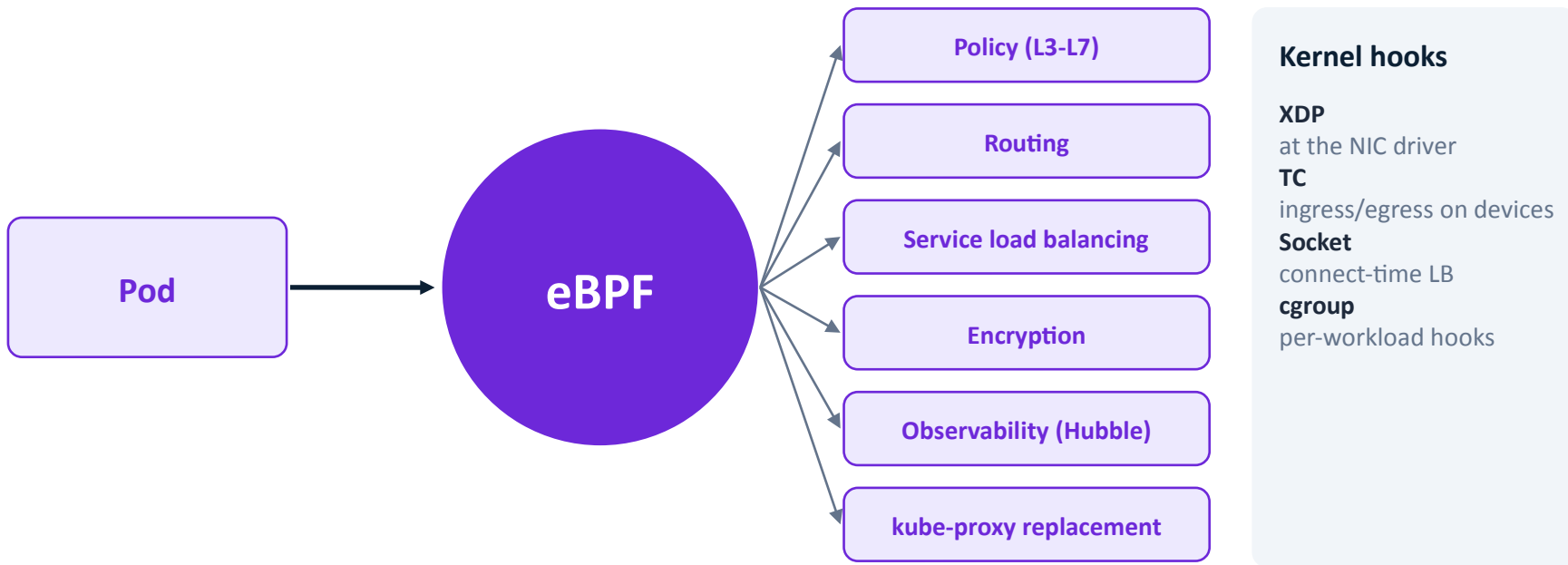
```
apiVersion: networking.k8s.io/v1
kind: NetworkPolicy
metadata:
  name: db-allow-backend
spec:
  podSelector:
    matchLabels: {app: database}
  ingress:
    - from:
      - podSelector:
          matchLabels: {app: backend}
      ports:
        - port: 5432
```

04

Cilium

An eBPF-based networking, security and observability platform

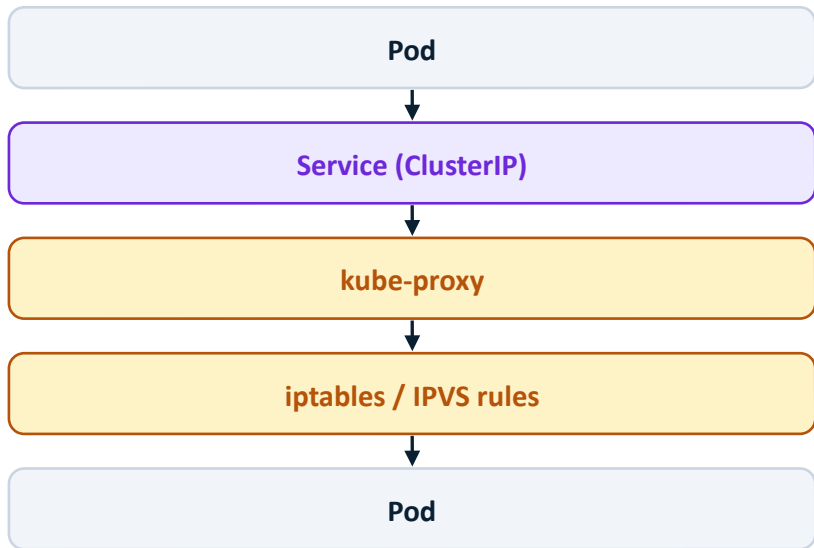
eBPF at the centre of the datapath



Tunnel mode (VXLAN/Geneve) or native routing; identity-based rather than IP-based policy.

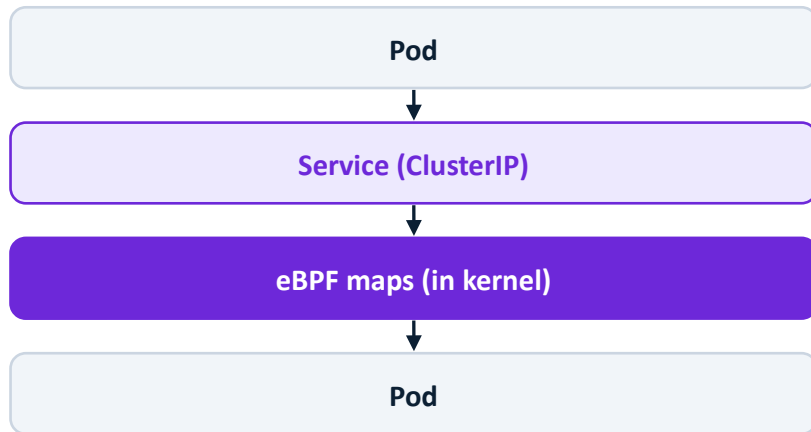
iptables + kube-proxy vs eBPF

Traditional



Rules scale linearly; long chains on large clusters

Cilium



Hash-table lookups; no kube-proxy DaemonSet needed

L7 policy and Hubble flow observability

Beyond "ALLOW TCP 8080"

```
apiVersion: cilium.io/v2
kind: CiliumNetworkPolicy
metadata: {name: api-read-only}
spec:
  endpointSelector:
    matchLabels: {app: backend}
  ingress:
    - fromEndpoints:
      - matchLabels: {app: frontend}
      toPorts:
        - ports: [{port: "8080", protocol: TCP}]
          rules:
            http:
              - {method: GET, path: "/api/*"}
```

Hubble answers

Who

source Pod / identity

What

HTTP GET, DNS query

Where

destination Pod / service

Port

TCP 8080

Verdict

FORWARDED / DROPPED

```
hubble observe --verdict DROPPED
```

Frontend may GET /api/; a POST is dropped at L7.*

Encapsulation is not encryption

VXLAN and IP-in-IP hide nothing: anyone on the underlay can read the inner packet.

Flannel

- vxlan: no encryption
- wireguard-native: encrypted

```
--flannel-backend=  
wireguard-native
```

Calico

- WireGuard (node-to-node)
- IPsec in commercial editions

```
FelixConfiguration:  
  wireguardEnabled
```

Cilium

- WireGuard
- IPsec

```
encryption.enabled + type
```

How each CNI hands out Pod IPs

Flannel / Canal

host-local

Fixed /24 per node from spec.podCIDR. Simple, but a small node wastes most of its range and a busy node can run out.

```
Node1 → 10.42.1.0/24  
Node2 → 10.42.2.0/24
```

Calico

Calico IPAM

IP pools split into small blocks (default /26) given to nodes on demand. Multiple pools, per-namespace pools.

```
Pool 10.42.0.0/16  
→ blocks of 64 IPs
```

Cilium

Cilium IPAM

cluster-pool, Kubernetes host-scope, and cloud modes such as AWS ENI and Azure IPAM.

```
cluster-pool  
eni / azure / multi-pool
```

How a remote Pod is reached

Technology	Method	Encapsulated?
Flannel vxlan	VXLAN overlay (UDP 8472)	Yes
Flannel host-gw	Linux routes to node IPs (L2 only)	No
Canal	Flannel routing + Calico policy	Yes (VXLAN)
Calico VXLAN	VXLAN overlay (UDP 4789)	Yes
Calico IP-in-IP	IP protocol 4 tunnel	Yes
Calico BGP	Native routing via BGP	No
Cilium tunnel	VXLAN or Geneve	Yes
Cilium native	Native routing, eBPF forwarding	No

Rule of thumb

Overlay

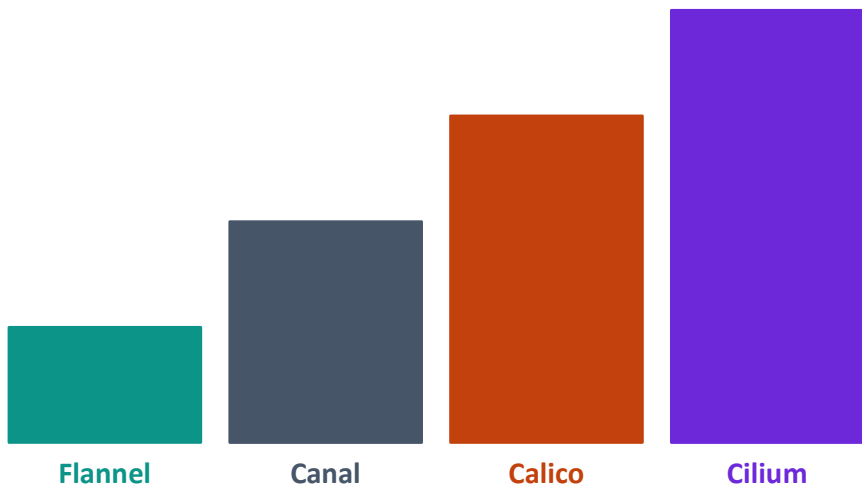
when you cannot change the underlay

Native routing

when you control the network and want full MTU and visibility

Advanced does not mean universally better

Capabilities + concepts to learn



Simple 3-node cluster, labs, edge

Flannel (K3s default)

Need policy, keep Flannel ops model

Canal (or K3s embedded policy)

BGP, advanced routing, enterprise policy

Calico

eBPF, no kube-proxy, L7, deep observability

Cilium

Running each option on K3s

```
# Flannel (default) - pick a backend
# (alternatives: host-gw, wireguard-native)
k3s server --flannel-backend=vxlan

# Bring your own CNI (Calico or Cilium)
k3s server --flannel-backend=none --disable-network-policy

# Cilium replacing kube-proxy as well
k3s server --flannel-backend=none --disable-network-policy \
  --disable-kube-proxy

# then install the CNI, e.g. with Helm
helm install cilium cilium/cilium -n kube-system ...
```

K3s-specific facts

- Flannel runs inside the k3s binary, not as a DaemonSet
- Embedded kube-router controller enforces NetworkPolicy with Flannel
- Disable it when another CNI enforces policy
- Canal is not bundled in K3s; it is RKE2's default

Build up one layer at a time

Each level makes the need for the next one obvious, instead of memorising flannel.yaml, canal.yaml, calico.yaml, cilium.yaml.



KEY TAKEAWAYS

Know what each one actually is

Flannel

is primarily a network fabric: simple Pod connectivity.

Canal

combines Flannel networking with Calico policy.

Calico

is a networking + policy platform with multiple datapaths.

Cilium

is an eBPF platform for networking, security, LB and observability.